

GPUMODE Lecture Study Notes: Lecture 46

Distributed GEMM

Core Problem the Lecture Addresses

The lecture explains how to extend high-performance single-GPU GEMM kernels into **distributed GEMM** kernels that execute one logical matrix multiplication across multiple GPUs. In the CUTLASS context, **distributed GEMM** is the API layer used to implement **tensor parallelism**: splitting a large neural network layer, especially a transformer linear layer, across several GPUs connected by a high-bandwidth scale-up interconnect such as NVLink.

The core problem is the following. A single matrix multiplication

$$D = AB$$

is already heavily parallelized inside one GPU using thread blocks, warps, tensor cores, shared memory, registers, and carefully tiled memory movement. However, modern large language models often contain matrices so large that a single GPU either cannot store the full operands or cannot deliver enough throughput alone. Therefore, we want multiple GPUs to collectively compute one GEMM.

The challenge is that moving from one GPU to many GPUs changes the granularity of parallelism:

single-GPU GEMM: parallel agents = SMs,

whereas

distributed GEMM: parallel agents = GPUs.

Once GPUs become the parallel agents, memory hierarchy and communication dominate the design. Each GPU has its own HBM and local caches, so there is no single global memory space with uniform latency and bandwidth. Communication between GPUs must go through NVLink or another interconnect.

The central trade-off is therefore:

extra parallelism from more GPUs versus extra communication between GPUs.

Distributed GEMM is fast only if the communication can be hidden behind computation or fused into the natural GEMM pipeline.

Required Background

Matrix Multiplication as Inner Products

For matrices

$$A \in \mathbb{R}^{M \times K}, \quad B \in \mathbb{R}^{K \times N}, \quad D \in \mathbb{R}^{M \times N},$$

the output element is

$$D_{m,n} = \sum_{k=0}^{K-1} A_{m,k} B_{k,n}.$$

A naive mental model assigns one parallel worker to one output element. That worker reads one row of A , one column of B , computes the inner product, and writes one scalar.

This exposes a huge amount of parallelism, but it is not efficient on modern hardware because each output element would reload the same rows and columns repeatedly. The GPU would spend too much time moving data and too little time using tensor cores.

Tiled GEMM

High-performance GEMM uses tiling. Instead of assigning one worker to one scalar, the computation is divided into output tiles. A tile of D depends on a tile of rows from A and a tile of columns from B .

For an output tile

$$D_{I,J},$$

where I is a block of rows and J is a block of columns, the computation is

$$D_{I,J} = \sum_{K_t} A_{I,K_t} B_{K_t,J}.$$

Here K_t represents a tile along the reduction dimension. This formulation captures the essence of optimized GEMM:

- load reusable tiles of A and B ;
- stage them through the memory hierarchy;
- reuse them many times in registers or shared memory;
- accumulate partial results;
- write output tiles.

The Contracting Dimension

The dimension K is the **contracting dimension** because it is summed over. Tiling along M and N creates independent output tiles. Tiling along K creates partial results that must later be reduced.

If K is split into P pieces, then each worker computes

$$D_{m,n}^{(p)} = \sum_{k \in K_p} A_{m,k} B_{k,n},$$

and the final output is

$$D_{m,n} = \sum_{p=0}^{P-1} D_{m,n}^{(p)}.$$

This reduction is possible because addition is associative:

$$x_0 + x_1 + x_2 + x_3 = (x_0 + x_1) + (x_2 + x_3).$$

This associativity is the mathematical property that makes reduce-scatter schedules possible.

Main Concepts

From GEMM to Distributed GEMM

A single-GPU GEMM distributes tiles among SMs. A distributed GEMM distributes larger tiles among GPUs. The analogy is:

Level	Parallel agents	Communication path
single GPU	SMs	L2, HBM, shared memory, registers
multi GPU	GPUs	NVLink, peer access, copy engines

In single-GPU GEMM, an SM has access to global memory and uses shared memory and registers for locality. In distributed GEMM, each GPU has its

own HBM, and remote GPU data must be explicitly communicated or accessed through peer memory.

Tensor Parallelism

Tensor parallelism splits one neural network layer across multiple GPUs. In transformer models, most compute comes from linear layers and attention GEMMs. A linear layer is

$$Y = XW,$$

where

$$X \in \mathbb{R}^{T \times H}, \quad W \in \mathbb{R}^{H \times H'}, \quad Y \in \mathbb{R}^{T \times H'}.$$

Here T is the number of tokens, H is the hidden dimension, and H' is the output feature dimension.

In tensor parallelism, X , W , or Y may be sharded across GPUs. The sharding choice determines the communication pattern.

All-Gather Plus GEMM

In an all-gather schedule, each GPU initially owns only a shard of one operand, commonly activations or weights. The GPUs first exchange pieces so that each GPU receives the operand shard needed for its local GEMM.

A typical all-gather plus GEMM pattern is:

$$\text{all-gather operand shard} \longrightarrow \text{local GEMM}.$$

For example, if activations are sharded along rows, each GPU may all-gather activation tiles so that it can compute a full tile of the output.

The exposed version is:

$$T_{\text{total}} = T_{\text{all-gather}} + T_{\text{GEMM}}.$$

The pipelined version breaks the all-gather and GEMM into stages:

$$T_{\text{total}} \approx T_{\text{stage 0 comm}} + \sum_s \max(T_{\text{GEMM},s}, T_{\text{comm},s+1}).$$

The goal is to hide later communication stages behind current GEMM stages.

GEMM Plus Reduce-Scatter

When the K -dimension is sharded, each GPU can compute partial outputs without first communicating. However, the partial outputs are not final. They must be reduced across GPUs.

The basic reduce-scatter pattern is:

$$\text{local partial GEMM} \longrightarrow \text{reduce-scatter output.}$$

If GPU p owns the p -th shard of the contracting dimension, it computes

$$D^{(p)} = A^{(p)} B^{(p)}.$$

The final output is

$$D = \sum_{p=0}^{P-1} D^{(p)}.$$

Reduce-scatter combines two operations:

1. reduce all partial outputs;
2. scatter the final output shards so that each GPU owns only its assigned slice.

Collective Communication

A collective all-gather or reduce-scatter has all GPUs participating in the same logical communication operation. The GPUs move together through the pipeline stages. For example, in collective all-gather, all GPUs acquire the same activation slice for a stage and then perform the corresponding GEMM.

Collective schedules are simple and powerful, but they usually have exposed communication at the beginning or end:

all-gather plus GEMM: exposed initial all-gather,

GEMM plus reduce-scatter: exposed final reduce-scatter.

Point-to-Point Communication

Point-to-point schedules replace global collectives with peer-to-peer communication. Instead of asking all GPUs to participate in one collective, each GPU communicates with a specific peer.

A point-to-point all-gather schedule can be implemented as a ring. GPU i sends or exposes its activation shard to GPU $i - 1$ or GPU $i + 1$, depending on the direction.

The goal is to start immediately with local data:

local GEMM on owned shard $\parallel$ copy next shard from peer.

Then the schedule rotates through the shards until all output tiles have been computed.

Point-to-Point Reduce-Scatter

Point-to-point reduce-scatter is more subtle. The goal is for each GPU to end up owning a final output slice, but partial results for that slice may be computed by other GPUs.

For a final output shard D_j , the full result is

$$D_j = \sum_{p=0}^{P-1} D_j^{(p)}.$$

A point-to-point schedule can pass partial sums around the ring. Each GPU computes its contribution and adds it to the partial sum it received from a peer.

If $S_j^{(r)}$ denotes the running partial sum after r reductions, then

$$S_j^{(0)} = D_j^{(p_0)},$$

$$S_j^{(r+1)} = S_j^{(r)} + D_j^{(p_{r+1})}.$$

After all GPUs have contributed,

$$S_j^{(P-1)} = D_j.$$

The clever part is that the addition can be fused into the GEMM epilogue by treating the incoming partial result as the GEMM epilogue source tensor C .

Hardware-Level Explanation

Single-GPU Memory Hierarchy

Inside one GPU, GEMM uses the hierarchy:

HBM $\rightarrow$ L2 cache $\rightarrow$ L1/shared memory $\rightarrow$ registers $\rightarrow$ tensor cores.

The purpose of tiled GEMM is to move data through this hierarchy once and reuse it many times. Arithmetic intensity is increased by reusing tiles of A and B .

Multi-GPU Memory Hierarchy

In distributed GEMM, each GPU has:

- its own HBM;
- its own L2 cache;
- its own SMs;
- its own shared memory and registers;
- peer access to other GPUs over NVLink, if enabled.

There is no uniform global HBM. Remote memory is accessible only through communication mechanisms such as peer-to-peer copies or remote loads.

The effective hierarchy becomes:

remote GPU HBM $\rightarrow$ NVLink $\rightarrow$ local GPU $\rightarrow$ local HBM/L2 $\rightarrow$ SM memory hierarchy.

Why NVLink Matters

Tensor parallelism requires frequent communication at layer granularity. The communication volume can be large, and it occurs inside the critical path of model execution. Therefore, the method is aimed at high-bandwidth scale-up topologies, especially all-to-all NVLink systems.

A simplified performance condition is:

$$T_{\text{comm}} \leq T_{\text{compute}}$$

for communication to be hidden. If the interconnect is slow or high-latency, communication dominates and tensor parallelism becomes inefficient.

This is why the lecture emphasizes any-to-any NVLink topology. Running the same schedules over lower-bandwidth or higher-latency interconnects such as

inter-node InfiniBand would require different schedules and would usually not be suitable for fine-grained tensor parallel GEMM.

Copy Engines and Peer Access

For point-to-point all-gather, the implementation can use peer-to-peer memory copies. These copies are handled through GPU copy engines and peer access rather than NCCL collectives.

The required setup is conceptually:

```
// Pseudocode for enabling peer access.
for (int src = 0; src < num_devices; ++src) {
    cudaSetDevice(src);
    for (int dst = 0; dst < num_devices; ++dst) {
        if (src == dst) continue;

        int can_access = 0;
        cudaDeviceCanAccessPeer(&can_access, src, dst);

        if (can_access) {
            cudaDeviceEnablePeerAccess(dst, 0);
        }
    }
}
```

Each GPU can then use remote pointers or peer copies to access memory located on another GPU.

Remote Epilogue Source

For reduce-scatter, the lecture highlights an important trick. GEMM already supports an epilogue of the form

$$D = \alpha AB + \beta C.$$

If C points to a remote partial result, then the GEMM epilogue can perform a reduction:

$$D_{\text{new}} = A_{\text{local}}B_{\text{local}} + D_{\text{remote partial}}.$$

This avoids launching a separate reduction kernel or explicit NCCL operation for that stage.

The epilogue load may be remote, but the GEMM kernel can overlap that load with the mainloop computation, so the additional cost can be small if the schedule is well balanced.

CUDA Streams and Why They Are Not Enough

CUDA streams execute operations in order within one stream. Different streams can run concurrently, but CUDA does not guarantee ordering across streams unless explicit synchronization is inserted.

A dangerous pattern is:

```
// Conceptual bad pattern.
stream0: cudaMemcpyAsync(buffer, peer_buffer, bytes);
stream0: cudaMemsetAsync(flag, 1, sizeof(int));

stream1: kernel_waiting_on_flag<<<...>>>(flag, buffer);
```

If the driver schedules the waiting kernel before the copy and flag update, the kernel may spin forever waiting for a signal that never executes. This violates the intended dependency structure.

Therefore, distributed GEMM uses CUDA graphs to express dependencies more robustly.

CUDA Graphs

CUDA graphs allow the program to describe a DAG of operations. A graph node can be:

- a kernel launch;
- a memcpy;
- a memset;
- a barrier or dependency node;
- a host node or other CUDA graph operation.

For point-to-point all-gather plus GEMM, a graph may contain:

$$\text{barrier} \rightarrow \begin{cases} \text{GEMM stage 0,} \\ \text{memcpy stage 1} \end{cases} \rightarrow \text{GEMM stage 1} \rightarrow \dots$$

Some dependencies may be explicit graph edges. Others may be implicit through arrival flags that kernels poll.

Programmatic Dependent Launch

Programmatic Dependent Launch, abbreviated PDL, is a Hopper-and-newer feature that helps overlap dependent kernels in the same stream.

Without PDL, the timeline is:

kernel A $\rightarrow$ launch gap $\rightarrow$ kernel B.

With PDL, kernel A can signal that kernel B is allowed to start before kernel A fully exits, as long as kernel B waits at the correct memory dependency point before consuming A's results.

The intended overlap is:

kernel A wind-down $\parallel$ kernel B ramp-up.

Persistent GEMM kernels often have nontrivial ramp-up time: initializing shared memory, barriers, TMA descriptors, warp-group roles, and pipeline state. PDL allows that setup to overlap with the previous kernel's final work.

Mathematical Reasoning

Communication Volume

Assume P GPUs and a tensor X sharded evenly across GPUs. Each GPU owns

$$\frac{|X|}{P}$$

elements.

A full all-gather causes each GPU to receive the other $P - 1$ shards:

$$V_{\text{recv per GPU}} = \frac{P - 1}{P} |X|.$$

Across all GPUs, total network traffic is approximately

$$V_{\text{total}} = (P - 1) |X|.$$

For reduce-scatter on an output tensor D , each GPU contributes partial results and receives only its final shard. The exact traffic depends on the algorithm, but the natural scale is also proportional to

$$\frac{P - 1}{P} |D|$$

per GPU.

Compute Cost of GEMM

The total floating point work for GEMM is

$$\text{FLOPs} = 2MNK.$$

The factor of 2 counts one multiply and one add per inner-loop element.

For P GPUs with perfect load balance, the ideal per-GPU compute work is

$$\text{FLOPs}_{\text{local}} = \frac{2MNK}{P}.$$

However, pipelining may split this local GEMM into stages. If there are S pipeline stages, then each local stage has roughly

$$\text{FLOPs}_{\text{stage}} = \frac{2MNK}{PS},$$

up to shape-dependent differences.

Arithmetic Intensity

Arithmetic intensity is

$$\text{AI} = \frac{\text{FLOPs}}{\text{bytes moved}}.$$

For a local GEMM

$$A_\ell \in \mathbb{R}^{M_\ell \times K_\ell}, \quad B_\ell \in \mathbb{R}^{K_\ell \times N_\ell}, \quad D_\ell \in \mathbb{R}^{M_\ell \times N_\ell},$$

the approximate arithmetic intensity is

$$\text{AI}_\ell = \frac{2M_\ell N_\ell K_\ell}{b_A M_\ell K_\ell + b_B K_\ell N_\ell + b_D M_\ell N_\ell},$$

where b_A , b_B , and b_D are bytes per element for A , B , and D .

This is only a first-order model. A real GEMM also has effects from cache reuse, epilogue loads, alignment, TMA behavior, shared memory pipelines, and tensor core instruction shapes.

Roofline Model

A kernel is compute-bound if

$$\text{AI} > \frac{F_{\max}}{B_{\max}},$$

where $F_{\max}$ is peak compute throughput and $B_{\max}$ is memory bandwidth.

A kernel is memory-bound if

$$\text{AI} < \frac{F_{\max}}{B_{\max}}.$$

In distributed GEMM, this model must be applied both locally and globally. A pipeline stage can become slower if local GEMMs become too small. Splitting too aggressively can reduce arithmetic intensity and cause memory bandwidth or launch latency to dominate.

Pipeline Stage Model

Let each stage have compute time C_s and communication time R_s . If communication and compute overlap perfectly after the first stage, then

$$T \approx C_0 + \sum_{s=1}^{S-1} \max(C_s, R_s) + T_{\text{tail}}.$$

For all-gather plus GEMM, there may be a small initial communication unless the first stage uses already-local data. For reduce-scatter, there may be a tail reduction unless the final reduction is fused into the epilogue.

The ideal point-to-point schedule tries to make

$$T_{\text{exposed comm}} \approx 0.$$

Why More Pipeline Stages Can Hurt

Increasing the number of stages improves overlap opportunities, but it also shrinks each local GEMM. If S becomes too large, then each stage may have poor arithmetic intensity:

$$\text{AI}_{\text{stage}} \downarrow \quad \text{as} \quad S \uparrow.$$

Also, communication latency becomes more visible:

$$T_{\text{comm},s} = T_{\text{latency}} + \frac{V_s}{B_{\text{link}}}.$$

If V_s is small, then T_{latency} dominates. In that regime, more stages can make the pipeline slower.

Code Walkthrough

Single-GPU GEMM Mental Model

A minimal CUDA GEMM kernel is not performance competitive, but it illustrates the mapping from threads to output elements.

```
__global__ void naive_gemm(
    const float* A,
    const float* B,
    float* D,
    int M,
    int N,
    int K) {

    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;

    if (row < M && col < N) {
        float acc = 0.0f;

        for (int k = 0; k < K; ++k) {
            acc += A[row * K + k] * B[k * N + col];
        }

        D[row * N + col] = acc;
    }
}
```

The indexing computes

$$\text{row} = b_y T_y + t_y, \quad \text{col} = b_x T_x + t_x.$$

Each thread computes one scalar. This is easy to understand but inefficient because it does not reuse A and B tiles through shared memory or registers.

Tiled GEMM Skeleton

A tiled kernel stages tiles of A and B into shared memory.

```

template <int TILE>
__global__ void tiled_gemm(
    const float* A,
    const float* B,
    float* D,
    int M,
    int N,
    int K) {

    __shared__ float As[TILE][TILE];
    __shared__ float Bs[TILE][TILE];

    int row = blockIdx.y * TILE + threadIdx.y;
    int col = blockIdx.x * TILE + threadIdx.x;

    float acc = 0.0f;

    for (int kt = 0; kt < K; kt += TILE) {
        int a_col = kt + threadIdx.x;
        int b_row = kt + threadIdx.y;

        As[threadIdx.y][threadIdx.x] =
            (row < M && a_col < K) ? A[row * K + a_col] : 0.0f;

        Bs[threadIdx.y][threadIdx.x] =
            (b_row < K && col < N) ? B[b_row * N + col] : 0.0f;

        __syncthreads();

        for (int kk = 0; kk < TILE; ++kk) {
            acc += As[threadIdx.y][kk] * Bs[kk][threadIdx.x];
        }

        __syncthreads();
    }

    if (row < M && col < N) {
        D[row * N + col] = acc;
    }
}

```

This kernel improves locality, but production GEMM uses tensor cores, warp-level tiling, register fragments, asynchronous copies, TMA on Hopper, and carefully scheduled mainloops. CUTLASS abstracts these details into reusable building blocks.

CUTLASS GEMM Construction Pattern

A CUTLASS GEMM is often assembled from:

- a mainloop collective;
- an epilogue collective;
- a GEMM kernel type;
- a device-level adapter.

Conceptually:

// Pseudocode structure inspired by CUTLASS 3.x.

```
using CollectiveMainloop = typename cutlass::gemm::collective::CollectiveBuilder<
    ArchTag,
    OpClass,
    ElementA,
    LayoutA,
    AlignmentA,
    ElementB,
    LayoutB,
    AlignmentB,
    ElementAccumulator,
    TileShape,
    ClusterShape,
    StageCount,
    KernelSchedule
>::CollectiveOp;

using CollectiveEpilogue = typename cutlass::epilogue::collective::CollectiveBuilder<
    ArchTag,
    OpClass,
    TileShape,
    ClusterShape,
    EpilogueTileType,
    ElementAccumulator,
    ElementCompute,
    ElementC,
    LayoutC,
    AlignmentC,
    ElementD,
    LayoutD,
    AlignmentD,
    EpilogueSchedule
>::CollectiveOp;

using GemmKernel = cutlass::gemm::kernel::GemmUniversal<
```

```

        ProblemShape,
        CollectiveMainloop,
        CollectiveEpilogue
    >;

using Gemm = cutlass::gemm::device::GemmUniversalAdapter<GemmKernel>;

```

The lecture's key point is that distributed GEMM tries to preserve this structure. Instead of rewriting GEMM kernels, the implementation wraps compatible CUTLASS kernels.

Distributed GEMM Schedule Selection

The distributed GEMM extension adds a schedule type. The schedule tells CUTLASS how tensors are sharded, how stages map to devices, and whether communication is all-gather-like or reduce-scatter-like.

Conceptually:

```

// Pseudocode only.
using DistributedSchedule =
    cutlass::experimental::distributed::P2PAllGatherGemm<
        TensorParallelSize
    >;

using DistributedGemmKernel =
    cutlass::experimental::distributed::kernel::DistributedGemmKernel<
        GemmKernel,
        DistributedSchedule
    >;

using DistributedGemm =
    cutlass::experimental::distributed::device::DistributedGemmUniversalAdapter<
        DistributedGemmKernel
    >;

```

The tensor parallel size P is static in the current API design for several implementation reasons. In the example discussed in the lecture, the default target is often $P = 8$, corresponding to an eight-GPU NVLink system.

Distributed GEMM Runtime Flow

At the user level, the goal is to make distributed GEMM look similar to a normal CUTLASS GEMM.

```

// Pseudocode for distributed GEMM usage.
DistributedGemm gemm;

```



```

DistributedGemm::Arguments args;
args.problem_shape = {M, N, K};
args.ptr_A = local_A_pointers;
args.ptr_B = local_B_pointers;
args.ptr_C = local_C_pointers;
args.ptr_D = local_D_pointers;
args.workspace = workspace_pointers;
args.tensor_parallel_rank = rank;
args.tensor_parallel_size = P;

size_t workspace_bytes = DistributedGemm::get_workspace_size(args);

gemm.initialize(args, workspace);
gemm.run(stream);

```

The actual implementation has more bookkeeping because the example uses one process controlling multiple GPUs. It sets up arrays of device-local objects, workspaces, local tensors, reference tensors, and peer-access metadata.

Peer-to-Peer All-Gather Stage

A simplified all-gather stage looks like this:

```

// Pseudocode for one point-to-point all-gather stage.
int peer = schedule.peer_for_stage(rank, stage);

cudaMemcpyPeerAsync(
    local_staging_buffer[stage],
    rank,
    remote_operand_buffer[peer],
    peer,
    bytes_for_stage,
    stream_for_graph_node
);

cudaMemsetAsync(
    arrival_flag[stage],
    1,
    sizeof(int),
    stream_for_graph_node
);

```

The memcpy pulls or pushes an operand tile through NVLink. The arrival flag signals that the data needed by a later GEMM stage is available.

The corresponding GEMM stage may poll the flag before starting the base CUTLASS GEMM.

Kernel Wrapper Logic

The kernel wrapper is intentionally small. It adds a few instructions before invoking the base CUTLASS GEMM kernel.

```
// Pseudocode for distributed kernel wrapper.
__global__ void distributed_gemm_kernel_wrapper(Params params) {
    if (params.use_pdl) {
        cudaTriggerProgrammaticLaunchCompletion();
    }

    if (params.must_wait_for_arrival) {
        while (*params.arrival_flag == 0) {
            // Poll local or peer-visible flag.
        }
    }

    if (params.use_pdl) {
        cudaGridDependencySynchronize();
    }

    BaseCutlassGemmKernel::operator()(params.base_gemm_params);
}
```

The real names differ, but the concept is accurate:

- optionally trigger PDL readiness;
- optionally wait for communication arrival;
- optionally synchronize dependent grids;
- execute the underlying CUTLASS GEMM.

The wrapper does not rewrite the GEMM mainloop. This is why the API can, in principle, support many CUTLASS kernels.

Reduce-Scatter Through GEMM Epilogue

For reduce-scatter, the GEMM epilogue can load a remote partial result as C :

```
// Conceptual epilogue computation.
for each output element in tile:
    partial = accumulator[element];
    previous = remote_C[element];
    D[element] = partial + previous;
```

Mathematically, this is:

$$D_{\text{stage}} = A_{\text{stage}}B_{\text{stage}} + C_{\text{remote partial}}.$$

The remote partial is not treated as a special communication primitive. It is simply the epilogue source pointer.

Performance Intuition

Parallelism Alone Is Not Enough

A key message from the lecture is that parallelism alone does not guarantee speed. Assigning one agent to each inner product is highly parallel, but inefficient. The agents must be fed enough work, and the memory system must supply data fast enough.

The real performance objective is:

keep tensor cores busy while hiding memory and communication.

When All-Gather Is Good

All-gather plus GEMM is attractive when the gathered operand can be reused heavily. If the cost of all-gathering X is amortized over a large GEMM, then communication can be hidden.

It is especially useful when the next operation naturally consumes the sharded output without additional communication.

When Reduce-Scatter Is Good

Reduce-scatter is attractive when K -dimension sharding lets all GPUs immediately start computing partial results. It avoids an initial all-gather, but it creates a final reduction requirement.

The point-to-point epilogue-source trick is powerful because it transforms explicit reduction into ordinary GEMM epilogue work.

Why Local GEMM Shape Matters

If tensor parallelism and pipeline staging make local GEMMs too small, performance drops. A local GEMM can become:

- memory-bandwidth-bound;
- latency-bound;
- launch-overhead-bound;
- tensor-core-underutilized;
- inefficient due to poor tile shapes.

A useful diagnostic is the local problem shape printed by the example. If the global GEMM is huge but each local stage is skinny or small, then the distributed schedule may not achieve high speed-of-light performance.

Speed of Light Interpretation

The lecture reports examples around H100 and B200 systems. The important interpretation is not only the absolute PFLOP/s number, but the ratio to the best achievable local GEMM. Distributed GEMM should be judged relative to:

$$\text{effective efficiency} = \frac{\text{distributed GEMM throughput}}{\text{best local GEMM throughput under comparable shapes}}.$$

If communication is well hidden, distributed GEMM can approach a large fraction of local GEMM performance. If communication or bad local shapes dominate, the efficiency falls.

Why Profiling Is Necessary

CUTLASS can generate many functionally equivalent GEMM kernels with different tile shapes, schedules, epilogues, and pipeline structures. Heuristics can narrow the search, but profiling is the reliable way to find the best kernel.

Distributed GEMM adds more dimensions:

- all-gather versus reduce-scatter;
- point-to-point versus collective;
- number of stages;
- operand sharding;
- output sharding;
- peer-access pattern;
- CUDA graph structure;
- PDL enabled or disabled;
- local GEMM kernel choice.

Therefore, distributed GEMM needs profiler support just as single-GPU GEMM does.

Common Misconceptions

Misconception: Distributed GEMM Is Just NCCL Plus GEMM

Distributed GEMM can use collectives, but the point-to-point implementation discussed in the lecture does not simply call NCCL. It uses peer access, memcpy, remote pointers, CUDA graphs, and CUTLASS kernel wrappers.

Misconception: More Pipeline Stages Always Improve Performance

More stages improve overlap only if the staged GEMMs remain large enough and the communication latency is hidden. If stages become too small, the pipeline may become slower.

Misconception: Any Multi-GPU Interconnect Is Good Enough

Tensor parallel GEMM requires high bandwidth and low latency because communication occurs inside individual layers. NVLink-scale bandwidth is the intended target. Inter-node networks require different schedules and are usually more appropriate for coarser-grained parallelism.

Misconception: The GEMM Kernel Must Be Completely Rewritten

The design goal is the opposite. Distributed GEMM should wrap existing CUTLASS 3.x kernels whenever possible. The distributed layer coordinates communication, graph dependencies, pointer movement, and schedule logic around the base GEMM.

Misconception: Streams Alone Are Sufficient for Correct Overlap

Multiple CUDA streams can create concurrency, but they do not guarantee ordering between dependent operations unless synchronization is explicit. CUDA graphs are safer for representing the required dependencies.

Practical Takeaways

- Distributed GEMM treats GPUs as the parallel agents, analogous to how single-GPU GEMM treats SMs as parallel agents.
- Sharding along M and N creates independent output tiles; sharding along K creates partial results that need reduction.

- All-gather plus GEMM moves operand shards before or during GEMM.
- GEMM plus reduce-scatter computes partial results first, then reduces and distributes the output.
- Point-to-point schedules can remove exposed collective communication by starting with local data and rotating data or partial sums.
- GEMM epilogues can be reused to perform reductions by treating remote partial results as the C tensor.
- CUDA graphs are used to coordinate concurrent copies, memset arrival signals, and GEMM kernels.
- Programmatic Dependent Launch reduces inter-kernel gaps by overlapping the next kernel's ramp-up with the previous kernel's wind-down.
- Local GEMM shape is critical. A bad local shape can destroy distributed performance even if the global problem is large.
- Profiling is mandatory because both the local GEMM kernel and the distributed schedule affect performance.

Project or Experiment Ideas

Experiment 1: Compare Single-GPU and Distributed GEMM Throughput

Run a large GEMM on one GPU and compare it against a tensor-parallel version across multiple GPUs. Measure:

$$\text{speedup} = \frac{T_{\text{single GPU}}}{T_{\text{distributed}}}.$$

Also compute scaling efficiency:

$$\text{scaling efficiency} = \frac{\text{speedup}}{P}.$$

Experiment 2: Vary Local GEMM Shape

Fix global compute size but vary sharding and pipeline stage count. Record local GEMM shapes and throughput. Identify where the local GEMM becomes memory-bound.

Experiment 3: All-Gather Versus Reduce-Scatter

Implement or benchmark two schedules:

all-gather plus GEMM,

and

GEMM plus reduce-scatter.

Compare communication volume, overlap, and final runtime.

Experiment 4: CUDA Graphs Versus Manual Streams

Construct a toy pipeline with peer memcopy, arrival flags, and kernels. Compare:

- multiple streams with explicit synchronization;
- CUDA graphs;
- CUDA graphs plus PDL on Hopper or newer.

Measure launch overhead and overlap.

Experiment 5: Remote Epilogue Source

Write a small kernel or use a GEMM epilogue that loads a remote buffer as the residual input. Compare against:

- separate GEMM plus separate reduction;
- GEMM with local epilogue source;
- GEMM with remote epilogue source.

Final Mental Model

Distributed GEMM is tiled GEMM lifted one level higher. On one GPU, tiles move through HBM, L2, shared memory, registers, and tensor cores. Across many GPUs, tiles and partial sums move through NVLink, peer memory, CUDA graph dependencies, copy engines, and GEMM epilogues.

The central design question is:

Which tensor shard should each GPU own, what communication is required to compute the next tile, and can that communication be hidden behind useful GEMM work?

All-gather schedules move operands so each GPU can compute complete output tiles. Reduce-scatter schedules compute partial outputs and then combine them. Point-to-point schedules try to avoid exposed collectives by rotating data or partial sums. CUTLASS distributed GEMM wraps high-performance local

GEMM kernels and uses schedule logic, peer access, CUDA graphs, and PDL to coordinate the multi-GPU execution.

The best distributed GEMM is not merely the one with the least communication. It is the one where communication volume, local GEMM shape, arithmetic intensity, kernel launch overhead, graph dependencies, and interconnect topology all line up so that tensor cores remain busy while data moves in the background.